

Day 11 – Design Patterns & Clean Architecture for GenAI Systems

1. Why architecture matters in GenAI applications

A basic RAG prototype may look like this:

```
question = input("Ask: ")

documents = vector_db.search(question)
prompt = build_prompt(question, documents)
answer = openai_client.generate(prompt)

print(answer)
```

This works for a demo. But production systems quickly become more complicated:

- Multiple LLM providers
- Multiple embedding models
- Keyword, vector, and hybrid retrieval
- Reranking
- Authentication and authorization
- Caching
- Retries and fallbacks
- Agent tools
- Prompt versioning
- Evaluation and observability
- Background ingestion
- Different configurations for development and production

Without good architecture, one file starts handling everything:

```
API request
+ authentication
+ retrieval
+ prompt building
+ model selection
+ retries
+ caching
+ logging
+ database access
+ response formatting
```

This creates tightly coupled code that is difficult to test, replace, or extend.

The architectural goal is:

| Separate what the system does from how external technologies perform it.

For example:

- The application needs to generate an answer.
- OpenAI, Anthropic, Gemini, or a local model may perform generation.
- The core business logic should not depend directly on one provider.

2. One consistent example

Throughout this lesson, imagine we are building a production knowledge assistant.

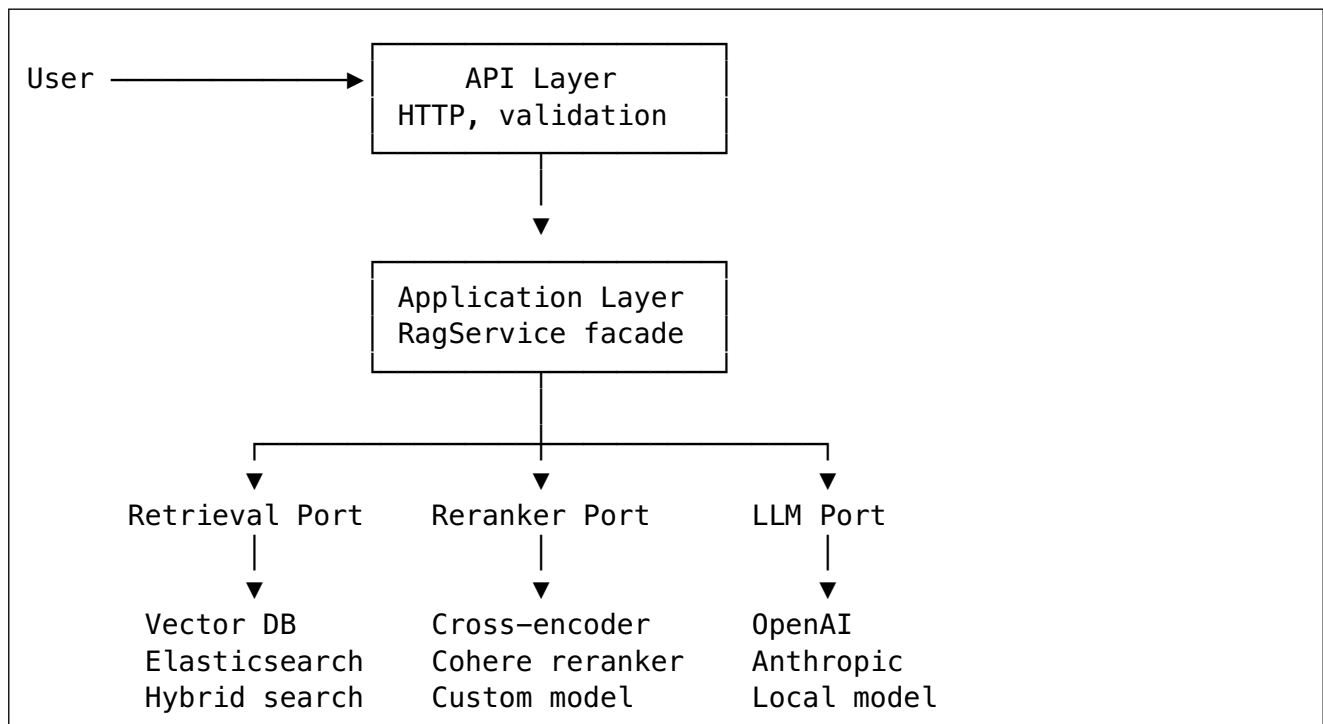
A user asks:

| “What is the refund policy for annual subscriptions?”

The system must:

1. Validate the request.
2. Retrieve relevant documents.
3. Rerank the retrieved documents.
4. Construct a grounded prompt.
5. Call an LLM.
6. Return an answer with citations.
7. Record latency, token usage, and retrieval quality.

A clean high-level architecture could look like this:



The application layer depends on interfaces such as LLM, Retriever, and Reranker.

External systems implement those interfaces through adapters.

3. Clean Architecture mental model

Clean Architecture separates code into areas with different responsibilities.

A practical GenAI version is:

API Layer HTTP routes, schemas, authentication
Application / Service Layer RAG workflow, agent workflow, use cases
Domain / Core Layer Interfaces, entities, business rules
Infrastructure / Data Layer LLM SDKs, vector DBs, SQL, Redis, queues

The dependency rule

Dependencies should generally point inward.

```

API —————> Application
Infrastructure —> Core interfaces
Application ———> Core interfaces

```

The core should not directly import provider-specific SDKs.

Bad dependency

```

# rag_service.py
from openai import OpenAI
from pinecone import Pinecone

```

Now the RAG service is directly coupled to OpenAI and Pinecone.

Better dependency

```

class LLM:
    def generate(self, prompt: str) -> str:
        ...

class Retriever:
    def retrieve(self, query: str) -> list[str]:
        ...

```

The service depends on these abstractions.

Infrastructure adapters implement them:

```

class OpenAILLM:
    ...

class PineconeRetriever:
    ...

```

4. SOLID principles for GenAI systems

SOLID is a set of five principles that help make software easier to maintain and extend.

They are not strict laws. They are design guidelines.

4.1 Single Responsibility Principle – SRP

| A class should have one primary reason to change.

A class should focus on one responsibility.

Poor design

```
class RagSystem:
    def load_documents(self):
        pass

    def chunk_documents(self):
        pass

    def create_embeddings(self):
        pass

    def store_embeddings(self):
        pass

    def retrieve_documents(self):
        pass

    def rerank_documents(self):
        pass

    def build_prompt(self):
        pass

    def call_llm(self):
        pass

    def log_metrics(self):
        pass
```

This class handles ingestion, retrieval, generation, storage, and monitoring.

A change to any of these areas affects the same class.

Better separation

DocumentLoader	→ loads documents
Chunker	→ splits documents
EmbeddingService	→ creates embeddings
DocumentRepository	→ stores and retrieves data
Retriever	→ finds candidate chunks
Reranker	→ reorders candidates
PromptBuilder	→ constructs prompts
LLMGateway	→ communicates with the model
RagService	→ coordinates the workflow

GenAI examples

- Prompt construction should not be inside the vector database class.
- Authentication should not be implemented inside the agent.
- Document parsing should not be part of the LLM provider adapter.
- API response formatting should not be inside retrieval logic.

4.2 Open/Closed Principle – OCP

| Software should be open for extension but closed for unnecessary modification.

You should be able to add new implementations without rewriting stable application logic.

Suppose the RAG service initially supports vector retrieval:

```
documents = vector_retriever.retrieve(query)
```

Later, you want:

- Keyword retrieval
- Hybrid retrieval
- Metadata-filtered retrieval
- Parent-child retrieval

The RAG service should not become:

```
if strategy == "vector":
    ...
elif strategy == "keyword":
    ...
elif strategy == "hybrid":
    ...
elif strategy == "parent_child":
    ...
```

Instead, define a retrieval interface:

```
from typing import Protocol

class RetrievalStrategy(Protocol):
    def retrieve(self, query: str, limit: int) -> list[str]:
        ...
```

Add implementations without modifying the main service:

```
class VectorRetrieval:
    def retrieve(self, query: str, limit: int) -> list[str]:
        return []

class HybridRetrieval:
    def retrieve(self, query: str, limit: int) -> list[str]:
        return []
```

4.3 Liskov Substitution Principle – LSP

| One implementation should be replaceable with another without breaking expected behavior.

Suppose every LLM provider implements:

```
class LLMPProvider:
    def generate(self, prompt: str) -> str:
        ...
```

Then this should work regardless of implementation:

```
def answer_question(llm: LLMPProvider, prompt: str) -> str:
    return llm.generate(prompt)
```

You should be able to pass:

- OpenAIAdapter
- AnthropicAdapter
- GeminiAdapter
- LocalModelAdapter

without changing `answer_question()`.

LSP violation example

Imagine one provider returns a string:

```
"Annual subscriptions can be refunded within 14 days."
```

Another returns a raw provider-specific dictionary:

```
{
  "candidates": [
    {
      "content": {
        "parts": [{"text": "Annual subscriptions..."}]
      }
    }
  ]
}
```

The second implementation does not respect the expected interface.

Adapters should normalize provider responses into the same domain object.

```
from dataclasses import dataclass

@dataclass
class GenerationResult:
    text: str
    input_tokens: int
    output_tokens: int
    model_name: str
```

Every provider should return `GenerationResult`.

4.4 Interface Segregation Principle – ISP

| Clients should not depend on methods they do not need.

Avoid one oversized interface:

```
class AIPlatform:
    def generate(self):
        ...

    def embed(self):
        ...

    def transcribe(self):
        ...

    def generate_image(self):
        ...

    def moderate(self):
        ...

    def fine_tune(self):
        ...
```

A RAG service may need only generation and embeddings.

Use smaller interfaces:

```
class TextGenerator:
    def generate(self, prompt: str) -> str:
        ...

class Embedder:
    def embed(self, texts: list[str]) -> list[list[float]]:
        ...

class Moderator:
    def moderate(self, text: str) -> bool:
        ...
```

This makes implementations and tests simpler.

4.5 Dependency Inversion Principle – DIP

| High-level business logic should depend on abstractions, not concrete infrastructure.

Tightly coupled version

```
class RagService:
    def __init__(self):
        self.llm = OpenAIClient()
        self.vector_db = PineconeClient()
```

The service decides its own concrete dependencies.

That makes testing difficult and tightly binds the service to those vendors.

Dependency injection version

```
class RagService:
    def __init__(self, retriever, reranker, llm):
        self.retriever = retriever
        self.reranker = reranker
        self.llm = llm
```

Production configuration can inject real implementations:

```
rag_service = RagService(
    retriever=PineconeRetriever(),
    reranker=CrossEncoderReranker(),
    llm=OpenAIAdapter(),
)
```

Tests can inject fake implementations:


```
rag_service = RagService(
    retriever=FakeRetriever(),
    reranker=FakeReranker(),
    llm=FakeLLM(),
)
```

This is extremely valuable for GenAI testing because unit tests should not call expensive external models.

5. Pattern comparison

Before studying each pattern, remember this distinction:

Pattern	Main purpose
Factory	Creates the correct object
Strategy	Selects an interchangeable algorithm
Adapter	Converts an external interface into your interface
Decorator	Adds behavior around an existing object
Facade	Provides a simple interface over a complex subsystem

A common production system may use all five together.

6. Factory Pattern

6.1 Intent

The Factory Pattern centralizes object creation.

Instead of spreading provider-selection logic across the application:

```
if provider == "openai":
    llm = OpenAIClient(...)
elif provider == "anthropic":
    llm = AnthropicClient(...)
```

put it in one factory.

6.2 Model-provider factory

```
from typing import Protocol

class LLM(Protocol):
    def generate(self, prompt: str) -> str:
        ...

class OpenAIAdapter:
    def generate(self, prompt: str) -> str:
        # Provider-specific SDK call would happen here.
        return f"OpenAI response for: {prompt}"

class AnthropicAdapter:
    def generate(self, prompt: str) -> str:
        # Provider-specific SDK call would happen here.
        return f"Anthropic response for: {prompt}"

class LocalModelAdapter:
    def generate(self, prompt: str) -> str:
        return f"Local model response for: {prompt}"

class LLMFactory:
    @staticmethod
    def create(provider: str) -> LLM:
        """Create the configured LLM implementation."""

        if provider == "openai":
            return OpenAIAdapter()

        if provider == "anthropic":
            return AnthropicAdapter()

        if provider == "local":
            return LocalModelAdapter()

        raise ValueError(f"Unsupported LLM provider: {provider}")
```

Usage:

```
llm = LLMFactory.create("openai")
answer = llm.generate("Explain hybrid retrieval.")
```

6.3 Where Factory applies in GenAI

Factories can create:

- LLM clients

- Embedding providers
- Vector databases
- Retrievers
- Rerankers
- Document loaders
- Agent tools
- Prompt builders
- Evaluation models

6.4 Factory trade-offs

Advantages

- Centralizes creation logic
- Hides provider-specific initialization
- Makes configuration-driven selection easy
- Reduces duplicated if/else blocks
- Simplifies testing

Risks

A large factory can become another giant conditional block:

```
if provider == ...
elif provider == ...
elif provider == ...
```

For many implementations, use a registry:

```
class LLMFactory:
    _providers = {
        "openai": OpenAIAdapter,
        "anthropic": AnthropicAdapter,
        "local": LocalModelAdapter,
    }

    @classmethod
    def create(cls, provider: str) -> LLM:
        provider_class = cls._providers.get(provider)

        if provider_class is None:
            raise ValueError(f"Unknown provider: {provider}")

        return provider_class()
```

7. Strategy Pattern

7.1 Intent

The Strategy Pattern makes algorithms interchangeable.

In RAG, retrieval is not always one fixed algorithm.

Possible strategies include:

```
Vector search
Keyword search
Hybrid search
Metadata-filtered search
Multi-query retrieval
Parent-child retrieval
Graph retrieval
```

The calling service should not need to know the internal details.

7.2 Retrieval strategy example

```
from typing import Protocol

class RetrievalStrategy(Protocol):
    def retrieve(self, query: str, limit: int) -> list[str]:
        ...

class VectorRetrievalStrategy:
    def retrieve(self, query: str, limit: int) -> list[str]:
        print("Running semantic vector search")
        return [
            "Vector result 1",
            "Vector result 2",
       ][:limit]

class KeywordRetrievalStrategy:
    def retrieve(self, query: str, limit: int) -> list[str]:
        print("Running keyword search")
        return [
            "Keyword result 1",
            "Keyword result 2",
       ][:limit]

class HybridRetrievalStrategy:
    def retrieve(self, query: str, limit: int) -> list[str]:
        print("Combining keyword and vector search")
        return [
            "Hybrid result 1",
            "Hybrid result 2",
       ][:limit]
```

The service accepts any strategy:

```

class RetrievalService:
    def __init__(self, strategy: RetrievalStrategy):
        self.strategy = strategy

    def search(self, query: str, limit: int = 5) -> list[str]:
        return self.strategy.retrieve(query, limit)

```

Usage:

```

retrieval_service = RetrievalService(
    strategy=HybridRetrievalStrategy()
)

documents = retrieval_service.search(
    query="annual subscription refund",
    limit=5,
)

```

7.3 Runtime strategy selection

The system may choose a strategy based on the query.

```

def choose_strategy(query: str) -> RetrievalStrategy:
    # Queries containing identifiers often benefit from keyword search.
    if "policy-" in query.lower():
        return KeywordRetrievalStrategy()

    # Natural-language questions often benefit from hybrid search.
    return HybridRetrievalStrategy()

```

For example:

```

Query: "Find policy-1942"
Strategy: Keyword search

Query: "Can I cancel my annual plan?"
Strategy: Hybrid search

```

7.4 Strategy for reranking

The same pattern applies to rerankers:

```

class RerankingStrategy(Protocol):
    def rerank(
        self,
        query: str,
        documents: list[str],
    ) -> list[str]:
        ...

```

Implementations:

NoOpReranker
CrossEncoderReranker
LLMReranker
CohereReranker
ReciprocalRankFusion

7.5 Strategy in agent systems

Agent strategies may include:

- ReAct execution
- Planner-executor
- Router agent
- Reflection loop
- Deterministic workflow
- Multi-agent collaboration

The application can choose an execution strategy according to the task's risk and complexity.

Simple FAQ	→ direct RAG
Document comparison	→ planner-executor
Sensitive operation	→ deterministic workflow + approval
Open-ended research	→ tool-using agent

7.6 Strategy trade-offs

Advantages

- Easy algorithm replacement
- Good support for A/B testing
- Simplifies retrieval experimentation
- Keeps algorithms independently testable
- Avoids large conditional workflows

Risks

- Too many tiny strategy classes
- Selection logic may become complicated
- Strategies may behave inconsistently
- Shared configuration can be duplicated

Use Strategy when algorithms are genuinely interchangeable, not merely to create more classes.

8. Adapter Pattern

8.1 Intent

An adapter translates an external system's interface into the interface your application expects.

Different LLM providers have different:

- Request structures

- Authentication mechanisms
- Message formats
- Streaming APIs
- Tool-calling formats
- Error types
- Token-usage responses

Your application should not handle those differences everywhere.

8.2 Domain interface

```
from dataclasses import dataclass
from typing import Protocol

@dataclass
class GenerationResult:
    text: str
    model_name: str
    input_tokens: int
    output_tokens: int

class LLM(Protocol):
    def generate(self, prompt: str) -> GenerationResult:
        ...
```

8.3 OpenAI adapter

```
class OpenAIAdapter:
    def __init__(self, client, model_name: str):
        self.client = client
        self.model_name = model_name

    def generate(self, prompt: str) -> GenerationResult:
        # The real call would use the provider's SDK.
        response = self.client.create(
            model=self.model_name,
            input=prompt,
        )

        # Convert the provider response into our standard result.
        return GenerationResult(
            text=response.output_text,
            model_name=self.model_name,
            input_tokens=response.usage.input_tokens,
            output_tokens=response.usage.output_tokens,
        )
```

8.4 Another provider adapter

```
class AnotherLLMAdapter:
    def __init__(self, client, model_name: str):
        self.client = client
        self.model_name = model_name

    def generate(self, prompt: str) -> GenerationResult:
        response = self.client.send_message(
            model=self.model_name,
            message=prompt,
        )

        # The external response is different, but the result returned
        # to our application is exactly the same type.
        return GenerationResult(
            text=response.content,
            model_name=self.model_name,
            input_tokens=response.input_token_count,
            output_tokens=response.output_token_count,
        )
```

The RAG service sees only:

```
result = llm.generate(prompt)
print(result.text)
```

It does not care which SDK produced the result.

8.5 Vector database adapters

Your core repository interface might be:


```

from dataclasses import dataclass
from typing import Protocol

@dataclass
class SearchResult:
    document_id: str
    text: str
    score: float
    metadata: dict

class VectorStore(Protocol):
    def search(
        self,
        vector: list[float],
        limit: int,
        filters: dict | None = None,
    ) -> list[SearchResult]:
        ...

```

Adapters could include:

```

PineconeVectorStoreAdapter
QdrantVectorStoreAdapter
PgVectorStoreAdapter
ElasticsearchVectorStoreAdapter

```

Each adapter converts its provider-specific response into `SearchResult`.

8.6 Adapter trade-offs

Advantages

- Prevents provider SDKs from leaking into business logic
- Makes vendor replacement easier
- Normalizes errors and responses
- Supports unit testing
- Reduces provider lock-in

Risks

- Lowest-common-denominator interfaces may hide powerful provider features
- Provider capabilities are not always identical
- Streaming and tool-calling abstractions can become complex
- Adapters require maintenance when provider APIs change

A useful approach is:

```

Common interface for common capabilities
+
Optional provider-specific extension for advanced features

```

Do not pretend every provider behaves identically when they do not.

9. Decorator Pattern

9.1 Intent

The Decorator Pattern adds behavior around an existing component without changing its core implementation.

Common GenAI decorators include:

- Logging
- Metrics
- Tracing
- Caching
- Authentication
- Rate limiting
- Retries
- Circuit breaking
- Content moderation
- Cost tracking

9.2 Logging decorator

Suppose the core LLM interface is:

```
class LLM:
    def generate(self, prompt: str) -> GenerationResult:
        ...
```

A logging decorator can wrap any implementation:

```

import time

class LoggingLLMDecorator:
    def __init__(self, wrapped_llm: LLM):
        self.wrapped_llm = wrapped_llm

    def generate(self, prompt: str) -> GenerationResult:
        start_time = time.perf_counter()

        try:
            result = self.wrapped_llm.generate(prompt)

            latency_ms = (time.perf_counter() - start_time) * 1000

            print(
                {
                    "event": "llm_call_completed",
                    "model": result.model_name,
                    "latency_ms": round(latency_ms, 2),
                    "input_tokens": result.input_tokens,
                    "output_tokens": result.output_tokens,
                }
            )

            return result

        except Exception as error:
            print(
                {
                    "event": "llm_call_failed",
                    "error": str(error),
                }
            )
            raise

```

9.3 Cache decorator

```
import hashlib

class CachedLLMDecorator:
    def __init__(self, wrapped_llm: LLM, cache):
        self.wrapped_llm = wrapped_llm
        self.cache = cache

    def generate(self, prompt: str) -> GenerationResult:
        # Avoid placing the raw prompt directly in the cache key.
        cache_key = hashlib.sha256(prompt.encode()).hexdigest()

        cached_result = self.cache.get(cache_key)

        if cached_result is not None:
            return cached_result

        result = self.wrapped_llm.generate(prompt)
        self.cache.set(cache_key, result)

        return result
```

9.4 Composing decorators

```
base_llm = OpenAIAdapter(
    client=openai_client,
    model_name="configured-model",
)

cached_llm = CachedLLMDecorator(
    wrapped_llm=base_llm,
    cache=redis_cache,
)

observable_llm = LoggingLLMDecorator(
    wrapped_llm=cached_llm,
)
```

The structure becomes:

```
Logging Decorator
  |
  ▼
Caching Decorator
  |
  ▼
OpenAI Adapter
```

9.5 Decorator order matters

Consider:

```
Logging(Cache(LLM))
```

This logs both cached and non-cached requests.

But:

```
Cache(Logging(LLM))
```

A cache hit may bypass the logging decorator.

Retries and caching also require careful ordering:

```
Retry(Cache(LLM))  
Cache(Retry(LLM))
```

These do not necessarily behave the same way.

9.6 Retry caution

Do not retry every failure blindly.

Retry transient failures such as:

- Temporary timeouts
- Rate-limit responses with backoff
- Temporary service unavailability
- Network interruptions

Do not repeatedly retry:

- Invalid API keys
- Invalid request schemas
- Context-window violations
- Content-policy rejection
- Unsupported model names

Use:

- Exponential backoff
- Jitter
- Maximum retry count
- Overall request deadline
- Idempotency controls for side-effecting tools

10. Facade Pattern

10.1 Intent

The Facade Pattern exposes a simple interface over a complicated subsystem.

A caller should not need to understand every RAG component.

Without a facade, an API route might do this:

```
query_vector = embedder.embed(question)
documents = vector_store.search(query_vector)
reranked = reranker.rerank(question, documents)
prompt = prompt_builder.build(question, reranked)
result = llm.generate(prompt)
citations = citation_builder.build(reranked)
```

This workflow leaks into the API layer.

Instead, expose a RagService.

10.2 RagService facade

```
from dataclasses import dataclass

@dataclass
class RagAnswer:
    answer: str
    citations: list[str]

class RagService:
    def __init__(
        self,
        retriever,
        reranker,
        prompt_builder,
        llm,
    ):
        self.retriever = retriever
        self.reranker = reranker
        self.prompt_builder = prompt_builder
        self.llm = llm

    def answer(self, question: str) -> RagAnswer:
        # Step 1: Retrieve more candidates than we finally need.
        candidates = self.retriever.retrieve(
            query=question,
            limit=20,
        )

        # Step 2: Keep the most relevant documents.
        selected_documents = self.reranker.rerank(
            query=question,
            documents=candidates,
            limit=5,
        )
```

```

# Step 3: Build a grounded prompt.
prompt = self.prompt_builder.build(
    question=question,
    documents=selected_documents,
)

# Step 4: Generate the final answer.
generation = self.llm.generate(prompt)

# Step 5: Return a stable application response.
return RagAnswer(
    answer=generation.text,
    citations=[
        document.source
        for document in selected_documents
    ],
)

```

The API route becomes simple:

```

@router.post("/questions")
async def ask_question(request: QuestionRequest):
    result = rag_service.answer(request.question)

    return {
        "answer": result.answer,
        "citations": result.citations,
    }

```

10.3 Facade for agent systems

An agent facade might expose:

```

class AgentService:
    async def execute(
        self,
        user_id: str,
        objective: str,
    ) -> AgentResult:
        ...

```

Internally, it may manage:

- State
- Planning
- Tool selection
- Tool authorization
- Checkpointing
- Retries
- Human approval
- Final answer generation

The API layer does not need to understand the graph or agent framework.

10.4 Facade trade-offs

Advantages

- Gives callers a simple API
- Prevents orchestration leakage
- Centralizes the use case
- Makes API handlers thin
- Simplifies integration tests

Risks

A facade can become a “god service” if it starts implementing all internal logic itself.

A good facade coordinates components. It does not replace all components.

11. Layered architecture for a GenAI application

A practical architecture often includes four main layers.

11.1 API layer

The API layer handles transport-related concerns.

Responsibilities:

- HTTP or gRPC endpoints
- Request validation
- Authentication
- Authorization
- Rate limiting
- Mapping HTTP requests to application commands
- Mapping application results to HTTP responses
- Status codes
- Streaming transport

Example:


```
@router.post("/rag/query")
async def query_rag(
    request: RagQueryRequest,
    service: RagService = Depends(get_rag_service),
):
    result = await service.answer(
        question=request.question,
        user_id=request.user_id,
    )

    return RagQueryResponse(
        answer=result.answer,
        citations=result.citations,
    )
```

The route should not:

- Query the vector database directly
- Construct prompts
- Select an LLM
- Execute agent tools
- Implement retry logic

Think of the API layer as a receptionist:

```
Receive request
Validate request
Call correct service
Return response
```

11.2 Service or application layer

This layer implements use cases.

Examples:

- AnswerQuestionUseCase
- IngestDocumentUseCase
- ExecuteAgentTaskUseCase
- SummarizeConversationUseCase
- EvaluateRagAnswerUseCase

Responsibilities:

- Coordinate workflow steps
- Apply application rules
- Call domain interfaces
- Manage transactions
- Handle use-case-level errors
- Enforce workflow policies

Example RAG workflow:

```
Validate query
  ↓
Check permissions
  ↓
Select retrieval strategy
  ↓
Retrieve documents
  ↓
Rerank documents
  ↓
Build prompt
  ↓
Generate answer
  ↓
Validate grounding
  ↓
Return answer and citations
```

11.3 Domain or core layer

This layer contains stable concepts and interfaces.

Examples:

```
Entities:
- Document
- Chunk
- Citation
- AgentTask
- ToolCall
- GenerationResult

Interfaces:
- LLM
- Embedder
- Retriever
- Reranker
- VectorStore
- Tool
- CheckpointRepository

Rules:
- Maximum number of retrieved documents
- Required citation policy
- Tool authorization policy
- Confidence threshold
```

This layer should contain minimal framework-specific code.

For example, a domain Document should not need to inherit from a Pinecone, LangChain, or web-framework class.

11.4 Infrastructure and data layer

This layer handles technology-specific details.

Examples:

- OpenAI or another provider SDK
- PostgreSQL
- Redis
- Qdrant
- Pinecone
- Elasticsearch
- Kafka
- S3-compatible storage
- External APIs
- LangChain integrations
- LlamaIndex integrations
- MCP clients or servers

Infrastructure classes implement core interfaces.

Core interface: LLM

Infrastructure implementation: OpenAIAdapter

Core interface: VectorStore

Infrastructure implementation: QdrantAdapter

Core interface: ConversationRepository

Infrastructure implementation: PostgresConversationRepository

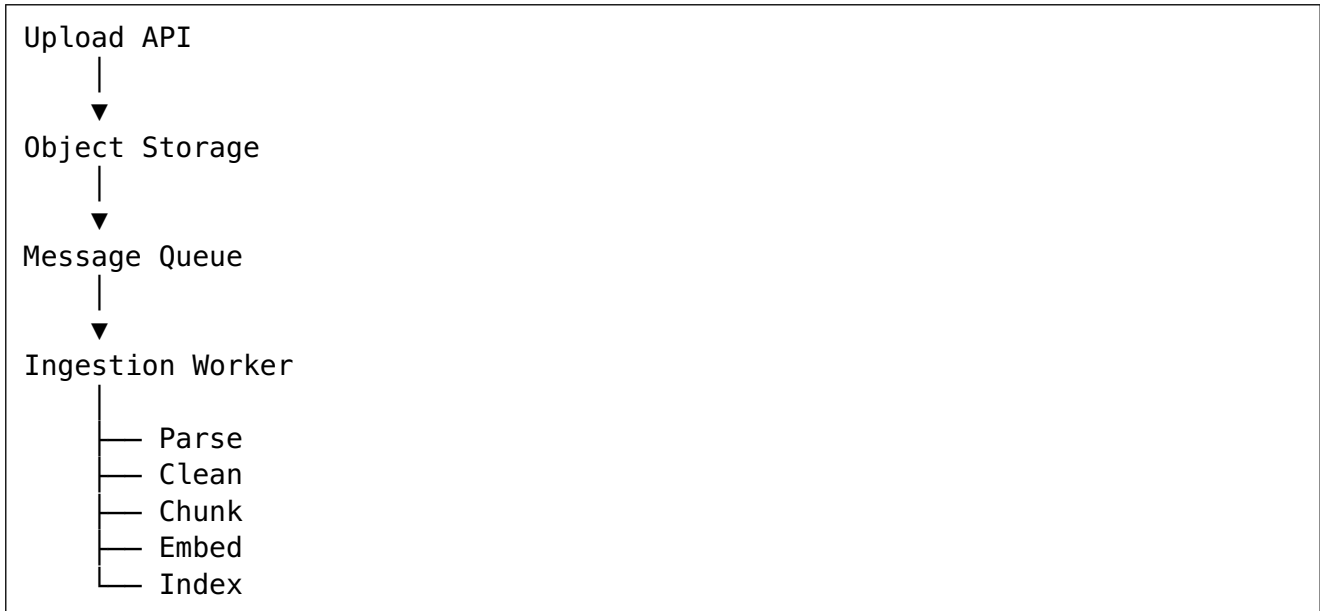
11.5 Worker layer

Long-running operations should usually run outside the request-response path.

Examples:

- PDF extraction
- Document chunking
- Embedding generation
- Bulk indexing
- Evaluation jobs
- Model batch inference
- Conversation summarization
- Stale-index cleanup

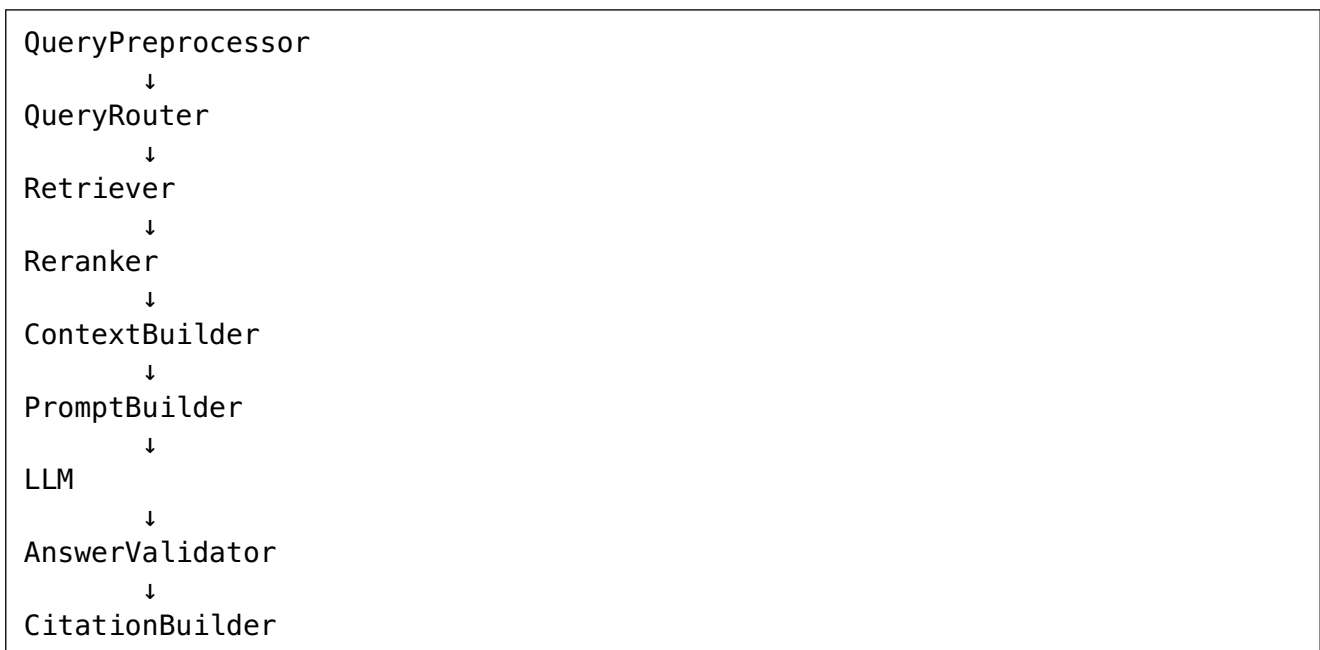
Architecture:



The upload endpoint should not keep an HTTP request open while processing a 500-page document.

12. Separation of concerns in a RAG pipeline

A well-separated RAG pipeline might contain:



Each component has a clear responsibility.

Example interfaces

```
class QueryPreprocessor:
    def process(self, query: str) -> str:
        ...

class Retriever:
    def retrieve(self, query: str, limit: int):
        ...

class Reranker:
    def rerank(self, query: str, documents: list, limit: int):
        ...

class PromptBuilder:
    def build(self, question: str, documents: list) -> str:
        ...

class AnswerValidator:
    def validate(self, answer: str, documents: list) -> bool:
        ...
```

This separation allows independent testing:

```
Retriever test:
Does it return relevant documents?

Reranker test:
Does it place the most relevant document first?

Prompt test:
Does it include the question and retrieved context?

Generation test:
Does the mocked LLM result flow correctly?

End-to-end test:
Does the complete system return a grounded answer?
```

13. Separation of concerns in agent systems

Agent code becomes difficult to maintain when planning, tool execution, permissions, state, and generation are mixed together.

A cleaner agent architecture is:

AgentService

- Planner
- Policy Engine
- Tool Registry
- Tool Executor
- State Repository
- Checkpointer
- Response Generator

Recommended separation

Planner

Determines the next step.

```
"Search the policy database."  
"Compare the two returned contracts."  
"Request human approval."
```

Tool registry

Knows which tools exist.

```
tool_registry = {  
    "search_policy": SearchPolicyTool(),  
    "fetch_customer": FetchCustomerTool(),  
    "create_refund": CreateRefundTool(),  
}
```

Policy engine

Determines whether a tool call is allowed.

Read-only search	→ automatically allowed
Customer data access	→ requires user scope
Financial action	→ requires approval
Destructive operation	→ prohibited or human-gated

Tool executor

Executes an approved tool call and normalizes its result.

State repository

Stores:

- Current task
- Previous messages
- Tool results
- Retry count
- Approval status
- Current workflow node

Checkpoint

Allows workflows to resume after interruption or failure.

14. Model-provider abstraction

A production model abstraction should usually cover more than one method call.

Possible capabilities include:

```
class ChatModel:
    async def generate(self, request):
        ...

    async def stream(self, request):
        ...

    async def generate_structured(self, request, schema):
        ...

    async def count_tokens(self, messages):
        ...
```

However, avoid forcing every provider into an overly broad interface.

A cleaner design may separate capabilities:

```
class TextGenerator:
    async def generate(self, request):
        ...

class StreamingTextGenerator:
    async def stream(self, request):
        ...

class ToolCallingModel:
    async def generate_with_tools(self, request, tools):
        ...

class StructuredOutputModel:
    async def generate_structured(self, request, schema):
        ...
```

This follows Interface Segregation.

Model routing

A routing strategy can select models based on requirements:

Simple classification
→ small, inexpensive model

Customer-facing grounded answer
→ stronger general-purpose model

Sensitive tool selection
→ model with reliable structured output

Offline summarization
→ batch or local model

Example:

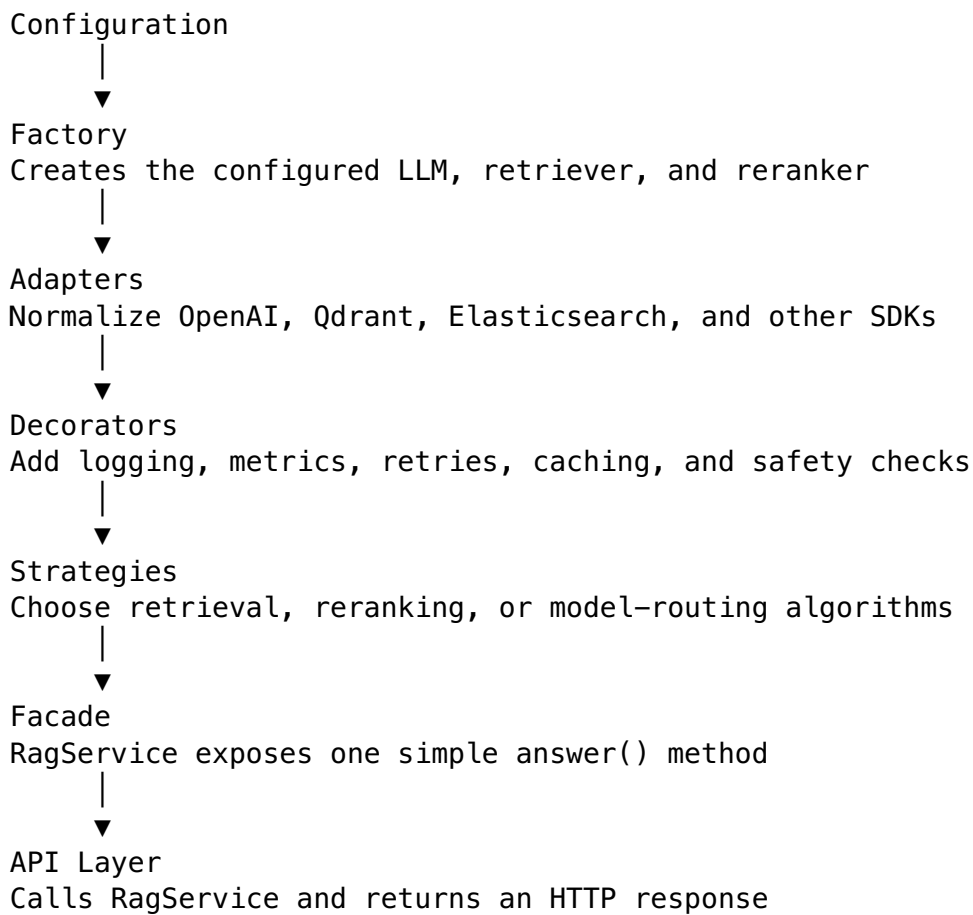
```
class ModelRoutingStrategy:
    def select_model(self, task):
        if task.requires_tools:
            return self.tool_calling_model

        if task.complexity == "low":
            return self.low_cost_model

        return self.high_quality_model
```

15. Putting all patterns together

Here is how the patterns can work in one system.



Example object composition:

```
def build_rag_service(settings):
    # Factory creates the provider-specific adapter.
    base_llm = LLMFactory.create(settings.llm_provider)

    # Decorators add cross-cutting behavior.
    observable_llm = LoggingLLMDecorator(base_llm)

    cached_llm = CachedLLMDecorator(
        wrapped_llm=observable_llm,
        cache=build_cache(settings),
    )

    # Strategy determines the retrieval algorithm.
    retrieval_strategy = RetrievalStrategyFactory.create(
        settings.retrieval_strategy
    )

    # Facade hides pipeline complexity.
    return RagService(
        retriever=retrieval_strategy,
        reranker=build_reranker(settings),
        prompt_builder=PromptBuilder(),
        llm=cached_llm,
    )
```

Notice that object construction occurs in one place.

This is sometimes called the:

- Composition root
- Bootstrap layer
- Dependency-wiring layer

The API route should not manually construct these objects for every request.

16. Example project structure

Here is a practical structure for a medium-sized GenAI application:

```
genai_app/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── dependencies.py
│   │   ├── exception_handlers.py
│   │   ├── middleware.py
│   │   └── routes/
│   │       ├── rag.py
│   │       ├── agents.py
│   │       ├── documents.py
│   │       └── health.py
│   ├── schemas/
│   │   ├── rag_requests.py
│   │   ├── rag_responses.py
│   │   ├── agent_requests.py
│   │   └── document_requests.py
│   ├── domain/
│   │   ├── entities/
│   │   │   ├── document.py
│   │   │   ├── chunk.py
│   │   │   ├── citation.py
│   │   │   └── agent_task.py
│   │   ├── interfaces/
│   │   │   ├── llm.py
│   │   │   ├── embedder.py
│   │   │   ├── retriever.py
│   │   │   ├── reranker.py
│   │   │   ├── vector_store.py
│   │   │   └── repositories.py
│   └── exceptions.py
```

```
— services/
  — rag_service.py
  — agent_service.py
  — ingestion_service.py
  — evaluation_service.py
  — authorization_service.py

— strategies/
  — retrieval/
    — vector.py
    — keyword.py
    — hybrid.py

  — reranking/
    — cross_encoder.py
    — reciprocal_rank.py

  — model_routing/
    — cost_based.py
    — capability_based.py

— infrastructure/
  — llms/
    — openai_adapter.py
    — anthropic_adapter.py
    — local_model_adapter.py

  — vector_stores/
    — qdrant_adapter.py
    — pgvector_adapter.py
    — elasticsearch_adapter.py

  — repositories/
    — postgres_conversation_repository.py
    — redis_checkpoint_repository.py

  — cache/
    — redis_cache.py

  — messaging/
    — kafka_publisher.py
    — task_queue.py

— agents/
  — state.py
  — planner.py
  — graph.py
  — policy.py
  — tools/
    — search_tool.py
    — customer_tool.py
```

```
└─ refund_tool.py

└─ prompts/
    └─ rag_answer_v1.txt
    └─ query_rewrite_v1.txt
    └─ agent_planner_v1.txt

└─ decorators/
    └─ logging.py
    └─ retry.py
    └─ caching.py
    └─ metrics.py

└─ factories/
    └─ llm_factory.py
    └─ retriever_factory.py
    └─ vector_store_factory.py

└─ workers/
    └─ ingestion_worker.py
    └─ embedding_worker.py
    └─ evaluation_worker.py

└─ configs/
    └─ settings.py
    └─ logging.py
    └─ model_registry.yaml

└─ bootstrap/
    └─ container.py

└─ tests/
    └─ unit/
        └─ test_rag_service.py
        └─ test_retrieval_strategies.py
        └─ test_prompt_builder.py

    └─ integration/
        └─ test_vector_store.py
        └─ test_llm_adapter.py

    └─ evaluation/
        └─ test_groundedness.py
        └─ test_context_recall.py
        └─ datasets/

    └─ end_to_end/
        └─ test_rag_api.py

└─ scripts/
    └─ ingest_documents.py
    └─ run_evaluation.py
```

```
|
|— migrations/
|— pyproject.toml
|— Dockerfile
|— README.md
```

17. Folder responsibilities

api/

Contains transport-specific code:

- Routes
- Middleware
- Dependency providers
- Exception mapping
- Authentication extraction

Keep business logic out of this directory.

services/

Contains application workflows:

- RAG answering
- Agent execution
- Ingestion coordination
- Evaluation coordination

Services should coordinate components rather than implement every detail.

domain/

Contains stable business concepts:

- Entities
- Interfaces
- Domain exceptions
- Policies and rules

This is the least technology-dependent layer.

infrastructure/

Contains external integrations:

- Model SDKs
- Vector databases
- SQL repositories
- Caches
- Message brokers

Provider-specific code belongs here.

strategies/

Contains interchangeable algorithms:

- Retrieval
- Reranking
- Routing
- Planning

workers/

Contains asynchronous background processing.

configs/

Contains validated configuration.

Do not scatter environment-variable access throughout the codebase.

Poor approach

```
api_key = os.getenv("OPENAI_API_KEY")
```

inside 20 different files.

Better approach

```
settings = Settings()
```

Create configuration once and inject it where required.

tests/

Separate:

- Unit tests
- Integration tests
- Evaluation tests
- End-to-end tests

Traditional tests check software correctness. Evaluation tests check AI behavior.

18. Testing clean GenAI architecture

Clean architecture makes different levels of testing possible.

18.1 Unit test

Test RagService using fake dependencies:

```

class FakeRetriever:
    def retrieve(self, query: str, limit: int):
        return [
            FakeDocument(
                text="Refunds are allowed within 14 days.",
                source="refund-policy.pdf",
            )
        ]

class FakeReranker:
    def rerank(self, query, documents, limit):
        return documents[:limit]

class FakeLLM:
    def generate(self, prompt):
        return GenerationResult(
            text="You can request a refund within 14 days.",
            model_name="fake-model",
            input_tokens=10,
            output_tokens=8,
        )

```

The test requires:

- No API key
- No internet
- No vector database
- No paid model call

18.2 Integration tests

Test one external boundary:

```

Qdrant adapter against a test Qdrant instance
OpenAI adapter against a controlled provider environment
PostgreSQL repository against a test database
Redis checkpoint adapter against test Redis

```

18.3 Evaluation tests

Measure AI-specific quality:

- Context precision
- Context recall
- Answer relevance
- Groundedness
- Citation correctness
- Tool-selection accuracy
- Tool success rate
- Structured-output validity

- Refusal correctness
- Latency and cost

18.4 End-to-end tests

Test:

```
HTTP request
  → service
  → retrieval
  → model
  → response
```

These tests are useful but slower, more expensive, and less deterministic.

Do not use only end-to-end tests.

19. Best practices

19.1 Keep framework objects at the edges

LangChain, LlamaIndex, LangGraph, and provider SDK objects should not unnecessarily spread through your entire domain model.

For example, avoid making every internal document a framework-specific document type.

Convert external objects at the boundary:

```
LlamaIndex Document
      ↓ Adapter/Mapper
Domain Document
```

This makes framework replacement easier.

19.2 Keep API routes thin

A route should generally:

```
Validate
Authorize
Call service
Map response
```

It should not contain the complete RAG pipeline.

19.3 Separate orchestration from component implementation

The RAG service should coordinate retrieval and generation.

The retriever should implement retrieval.

The prompt builder should build prompts.

The LLM adapter should communicate with the model.

19.4 Use dependency injection

Inject:

- LLM
- Retriever
- Reranker
- Cache
- Repositories
- Tool registry
- Policy engine

This improves testability and configurability.

19.5 Define stable domain models

Normalize external responses.

Examples:

```
GenerationResult
SearchResult
ToolResult
AgentState
Citation
```

Do not expose raw provider dictionaries to every layer.

19.6 Treat prompts as versioned application assets

Avoid large prompts embedded across business logic.

Use:

```
prompts/
├── rag_answer_v1.txt
├── rag_answer_v2.txt
└── query_rewrite_v1.txt
```

Record prompt version in traces and evaluation results.

19.7 Separate configuration from code

Configurations may control:

- Provider
- Model name
- Temperature
- Timeout

- Retrieval limit
- Reranking limit
- Similarity threshold
- Retry policy
- Cache TTL
- Feature flags

Validate configuration at startup.

19.8 Create a composition root

Construct and wire dependencies in one location:

```
bootstrap/container.py
```

Avoid constructing database and model clients throughout the application.

19.9 Design explicit error types

Normalize infrastructure errors:

```
class ModelTimeoutError(Exception):
    pass

class ModelRateLimitError(Exception):
    pass

class RetrievalUnavailableError(Exception):
    pass

class InvalidStructuredOutputError(Exception):
    pass
```

The service layer should not need to understand every provider's exception hierarchy.

19.10 Make side effects explicit in agents

Tools should clearly declare:

- Name
- Input schema
- Output schema
- Read-only or write operation
- Required authorization
- Retry safety
- Approval requirement

Example:

```
Tool: search_policy
Effect: read-only
Approval: not required
Retry: safe

Tool: issue_refund
Effect: financial write
Approval: required
Retry: only with idempotency key
```

20. Common mistakes

Mistake 1: Creating unnecessary abstractions

Not every small function needs an interface, factory, strategy, and adapter.

Overengineering can produce:

- Too many files
- Difficult navigation
- Empty wrapper classes
- Indirection without value

Create an abstraction when you expect:

- Multiple implementations
 - External dependency replacement
 - Independent testing
 - Clear business boundaries
 - Meaningful variation
-

Mistake 2: One giant RagService

A facade should coordinate components.

It should not contain:

- SQL queries
- Vector database SDK calls
- Prompt templates
- Model request parsing
- Logging implementation
- Authentication rules
- Document parsing

That would merely move the monolith into one class.

Mistake 3: Provider-specific objects leaking everywhere

For example:

```
def process_response(response: OpenAIResponse):  
    ...
```

inside the service layer.

This creates vendor coupling.

Convert provider responses into domain models inside adapters.

Mistake 4: Using inheritance when composition is simpler

Avoid deep hierarchies such as:

```
BaseModel  
├── CloudModel  
│   └── ChatModel  
│       └── ToolCallingModel  
│           └── OpenAIToolCallingModel
```

Prefer composition and small interfaces.

Mistake 5: Retrying unsafe agent tools

A model call can often be retried safely.

A payment, booking, deletion, or refund tool may not be safe to retry without idempotency.

```
First call succeeds  
Network response is lost  
Retry creates a duplicate action
```

Always classify tool side effects.

Mistake 6: Putting business logic in API routes

This makes logic difficult to:

- Reuse from workers
- Reuse from CLI applications
- Unit test
- Expose through another transport

Put the use case in the service layer.

Mistake 7: Treating every model provider as identical

Providers differ in:

- Context windows
- Tool calling
- Streaming

- Structured output
- Safety behavior
- Token counting
- Error handling

Use a common abstraction, but preserve capability differences honestly.

Mistake 8: Abstracting too early

A useful rule is:

| First understand what changes; then abstract the changing boundary.

Do not build a universal AI framework before the application requirements are clear.

Mistake 9: Mixing ingestion and online querying

Ingestion and serving have different performance characteristics.

Ingestion: CPU-heavy, asynchronous, batch-oriented
Query serving: Latency-sensitive, user-facing, availability-critical

Separate them operationally and architecturally.

Mistake 10: Ignoring observability interfaces

Observability should capture:

- Request ID
- User or tenant ID where permitted
- Model and prompt version
- Retrieval strategy
- Retrieved document IDs
- Retrieval scores
- Model latency
- Token usage
- Estimated cost
- Tool calls
- Errors and retries

Do not log confidential prompts or documents without appropriate controls.

21. Trade-offs of clean architecture

Advantages

- Better testability

- Easier provider replacement
- Clear separation of responsibilities
- Easier team ownership
- More controlled production changes
- Easier experimentation
- Reduced vendor coupling
- Better support for multiple deployment environments

Costs

- More files and interfaces
- Additional indirection
- Higher initial development effort
- More dependency wiring
- Risk of overengineering
- Developers must understand boundaries

Practical rule

For a small prototype:

```
API
RagService
Retriever
LLM client
```

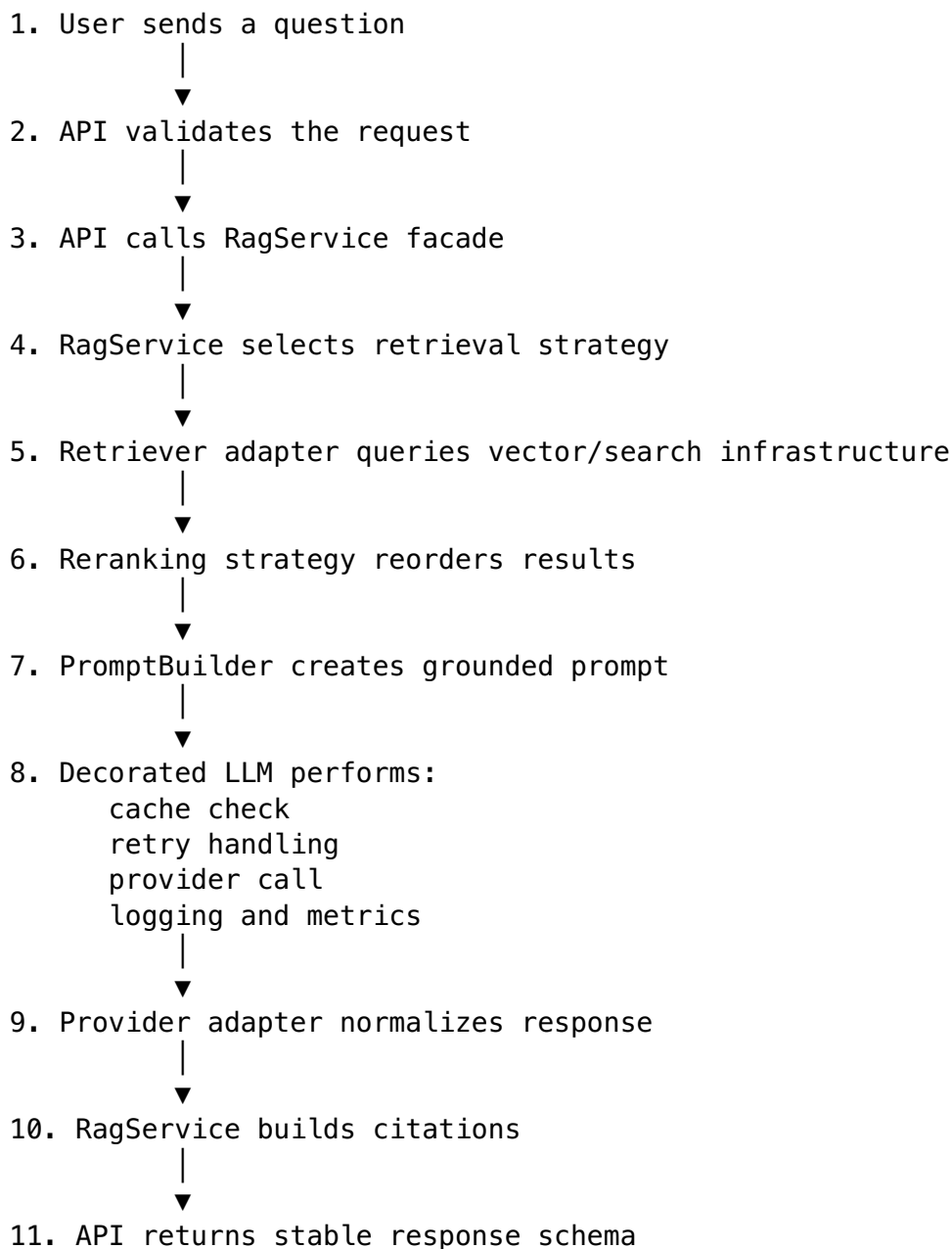
may be sufficient.

For a production platform with multiple providers, teams, agents, and data sources, stronger boundaries become valuable.

Architecture should grow with actual complexity.

22. End-to-end request flow

Here is the complete flow for our knowledge assistant:



The individual patterns have different roles:

Factory	→ Which LLM or retriever object should be created?
Strategy	→ Which retrieval or reranking algorithm should run?
Adapter	→ How do we normalize the provider's API?
Decorator	→ How do we add caching, retries, or logging?
Facade	→ How do we expose the whole pipeline simply?

23. Interview Q&A

1. What problem does the Factory Pattern solve in a GenAI system?

It centralizes object creation. For example, it can create the correct LLM adapter based on configuration without spreading provider-selection logic throughout the application.

2. How is Strategy different from Factory?

A Factory creates an object. A Strategy represents an interchangeable algorithm.

Example:

Factory:
Create an OpenAI or local-model client.

Strategy:
Choose vector, keyword, or hybrid retrieval.

3. Why use an Adapter for LLM providers?

Different providers have different request and response formats. An adapter converts those provider-specific formats into a stable application interface such as `GenerationResult`.

4. Where would you use the Decorator Pattern in RAG?

For cross-cutting behavior such as:

- Logging
- Metrics
- Caching
- Retries
- Tracing
- Rate limiting
- Safety checks

The decorator wraps an LLM or retriever without modifying its implementation.

5. What is the role of a RagService facade?

It provides a simple method such as `answer(question)` over a complex pipeline involving retrieval, reranking, prompt construction, generation, and citations.

6. How does Dependency Inversion help a GenAI service?

The service depends on interfaces such as `LLM` and `Retriever` instead of concrete classes such as an OpenAI SDK client or Pinecone client. This improves testing and makes providers easier to replace.

7. What should belong in the API layer?

The API layer should handle:

- Request parsing
- Schema validation
- Authentication
- Authorization

- HTTP status codes
- Calling application services
- Response serialization

It should not implement retrieval or prompt-building logic.

8. How would you test a RAG service without calling a real LLM?

Inject fake implementations of the LLM, retriever, and reranker. Then test orchestration and response construction deterministically without network calls or API costs.

9. What is a common architecture mistake in agent systems?

Mixing planning, tool execution, permissions, state management, and provider calls in one agent class. These responsibilities should be separated into components such as planner, policy engine, tool registry, executor, and state repository.

10. When is clean architecture too much?

It may be excessive for a small experiment with one model and one retrieval implementation. Introduce abstractions when there are real changing boundaries, multiple implementations, testing needs, or operational complexity.

24. Final revision map

SOLID

- SRP → One clear responsibility
- OCP → Extend using new implementations
- LSP → Implementations remain interchangeable
- ISP → Prefer small capability interfaces
- DIP → Depend on abstractions

Patterns

- Factory → Object creation
- Strategy → Algorithm selection
- Adapter → Interface conversion
- Decorator → Additional behavior
- Facade → Simplified subsystem API

Architecture

- API → Transport concerns
- Services → Use-case orchestration
- Domain/Core → Stable concepts and interfaces
- Infrastructure → Providers, databases, queues
- Workers → Asynchronous processing

The most important interview takeaway is:

Keep GenAI workflow logic independent from model providers, vector databases, web frameworks, and orchestration frameworks. Depend on stable interfaces, connect external technologies through adapters, and keep the complete use case behind a simple service facade.